03

# Por que a IA quebra seu código

Você pede uma correção, a IA corrige, funciona. Duas sessões depois, um bug novo aparece em outro lugar — e a sensação é sempre a mesma: consertou uma coisa, quebrou outra. A conclusão apressada é "a IA ainda não serve para manutenção". Este material mostra por que essa conclusão está errada — e o que realmente falta.

A IDEIA CENTRAL DESTE MATERIAL

**A IA não erra porque é incapaz. Ela erra porque está cega: vê só o fragmento que você colou, num sistema cheio de dependências invisíveis. O problema não é inteligência — é ausência de contexto. E contexto se constrói.**

## 1 O ciclo que todo desenvolvedor conhece

A cena se repete em qualquer time que usa IA no dia a dia:

```
Sessão 1: cola um bug → IA corrige → funciona → segue em frente
Sessão 2: bug novo em outro lugar → cola → IA corrige → funciona
Sessão 3: mais um bug — com jeito de ter sido causado pela correção anterior
Sessão 4: você passa mais tempo corrigindo o que a IA
          introduziu do que construindo features novas
```

Esse ciclo tem nome: **whack-a-mole** — o jogo de martelar a toupeira que sempre reaparece em outro buraco. E a explicação que a maioria dá para ele — "a IA não é boa o suficiente para manutenção" — atribui ao modelo um defeito que, na verdade, está no processo.

## 2 Cega, não burra: o que o modelo não vê

Quando você abre uma sessão nova e cola um trecho de código, o modelo enxerga aquele fragmento — e absolutamente nada além dele. Ele não sabe:

- Como o resto do sistema está estruturado e como os módulos se conectam
- Quais convenções o time definiu, e quais padrões já foram tentados e descartados
- Por que cada decisão de arquitetura foi tomada — e o que não pode mudar por restrição de negócio
- Qual é a stack completa, com versões específicas

- Quais partes do sistema são frágeis ou têm histórico de problemas

Com essas lacunas, o modelo faz o que qualquer engenheiro competente faria no lugar dele: **toma a decisão mais razoável com a informação disponível**. E uma decisão razoável, tomada às cegas num sistema com dependências invisíveis, quase sempre tem efeitos colaterais — que você só descobre quando testa.

> O bug não nasceu da incompetência do modelo. Nasceu da **ausência do contexto** que permitiria prever os efeitos colaterais da mudança. Lembre do material 01: sem contexto, os vetores vão para o centro do mapa — a resposta média, não a resposta do *seu* sistema.

## 3 Gerar é o problema fácil. Manter é o difícil.

Há uma assimetria que explica por que as demos de IA impressionam e o dia a dia frustra:

| | GERAR CÓDIGO NOVO | MANTER CÓDIGO EXISTENTE |
|---|---|---|
| **Ponto de partida** | Tela em branco | Sistema com história |
| **O que o modelo precisa** | Padrões gerais do treinamento | Contexto específico do sistema |
| **Estado a respeitar** | Nenhum | Decisões, convenções, partes frágeis |
| **Resultado sem contexto** | Tecnicamente correto | Decisão razoável com efeitos colaterais |

E aqui está a ironia: manutenção é o caso de uso que **mais exige contexto** — e é exatamente onde as pessoas **menos fornecem**. Cola-se o erro e espera-se a correção. Ninguém cola o arquivo inteiro, explica a arquitetura, descreve as convenções, conta o histórico da decisão que gerou o bug. O resultado é previsível.

## 4 O segredo dos produtos que “simplesmente funcionam”

Ferramentas como bolt.diy, Cursor e GitHub Copilot geram código bonito e consistente com prompts simples. A tentação é achar que usam um modelo melhor que o seu. Não usam — muitas vezes é **exatamente o mesmo modelo** que você acessa no chat.

A diferença é o **system prompt**: um conjunto extenso de instruções e contexto que chega ao modelo *antes* do seu pedido — definindo padrões de nomenclatura, estrutura de componentes, tratamento de erros, o que nunca fazer. Refinado ao longo de meses de uso real, ele é o ativo mais valioso desses produtos. Não o modelo, não a interface: **o contexto estruturado**.

```
Sem system prompt:
[seu prompt] → modelo → resultado genérico

Com system prompt:
[contexto rico] + [seu prompt] → modelo → resultado específico
```

## 5 Por que isso não resolve o SEU projeto

Um system prompt genérico resolve o que é igual em todos os projetos: estilo de resposta, formato de output, boas práticas gerais. Por isso essas ferramentas geram bem. Mas manutenção depende do que é **único** do seu projeto:

| O QUE SÓ O SEU PROJETO TEM | O QUE ACONTECE SE O MODELO NÃO SOUBER |
|---|---|
| **Decisões de arquitetura** — por que SQLite e não PostgreSQL, por que esse padrão de autenticação | O modelo reverte decisões deliberadas sem perceber |
| **Convenções do time** — nomenclatura, estrutura, onde ficam os testes | Cada correção introduz inconsistências que viram dívida técnica |
| **Histórico de problemas** — o que já quebrou, o que foi tentado e descartado | O modelo repete abordagens que já falharam |
| **Restrições de negócio** — o que não pode mudar, mesmo havendo solução "melhor" | Soluções tecnicamente elegantes que violam regras que você nunca escreveu |

Nenhum system prompt genérico carrega isso — porque é específico de cada projeto e **muda com o tempo**. A única saída é construir o contexto do projeto de forma estruturada e persistente.

## 6 Harness: o system prompt que cresce com o projeto

É exatamente isso que chamamos de **Harness**: o sistema de contexto vivo do projeto. Não um texto escrito uma vez, mas um conjunto de artefatos versionados que cresce a cada sessão e é injetado automaticamente em cada conversa nova.

**✗ SYSTEM PROMPT GENÉRICO**

Escrito uma vez, vale para qualquer projeto.

Sabe gerar código bem **em geral** — não sabe nada sobre o seu sistema, suas decisões, seu histórico.

**✓ HARNESS DO PROJETO**

Cresce com cada sessão, específico do seu sistema.

Sabe a stack, as convenções, as decisões, o histórico de problemas — e carrega tudo isso automaticamente.

**▸ A DIFERENÇA NA PRÁTICA**

Sem Harness, cada sessão de IA é um **novo contratado brilhante no primeiro dia de trabalho** — todos os dias. Talentoso, mas sem conhecer nada do sistema. Com Harness, é um **engenheiro sênior que está no projeto há dois anos**: conhece as decisões, os pontos frágeis e o que nunca se deve tocar.

## 7 Os artefatos do Harness — e onde estudar cada um

Cada artefato resolve uma classe específica de problema de contexto. Esta tabela é o mapa do restante da trilha:

| ARTEFATO | PERGUNTA QUE RESOLVE | MATERIAL |
|---|---|---|
| `SPEC.md` | O que construir? Problema, funcionalidades, critérios de aceitação. | 05 |
| `CLAUDE.md` / `AGENTS.md` | Como trabalhar neste projeto? Stack, convenções, regras de ouro. | 05 |
| `.claudeignore` | O que o agent nunca deve ler? Protege contexto e segredos. | 06 |
| `PLAN.md` | Em que ordem, com quais critérios? Fases, tasks e gates de teste. | 07 |
| `Agents` | Quem executa o quê? Escopo, tools e system prompt por domínio. | 09–10 |
| `Hooks` | O que deve acontecer sempre, independente do que o agent decidir? | 11–12 |
| `MCP` | Quais ferramentas externas o agent acessa de forma estruturada? | 13 |
| `RAG` | O que foi aprendido nas sessões anteriores? | 14 |

## 8 O ciclo, antes e depois

```
SEM HARNESS
Sessão 1: você explica tudo → agent implementa → sessão encerra → contexto perdido
Sessão 2: explica de novo → agent assume diferente → inconsistência
Sessão 3: bug → cola o fragmento → correção às cegas → novo bug
Sessão 4: whack-a-mole

COM HARNESS
Sessão 1: contexto carrega automaticamente → implementação consistente
Sessão N: bug → agent lê SPEC, CLAUDE.md, Rules → corrige conhecendo
          as dependências → sem efeitos colaterais surpresa
RAG:      "este módulo tem histórico do problema X, resolvido assim,
          não tentar Y" → o erro do passado não se repete
```

Com o tempo, o Harness acumula mais contexto sobre o sistema do que qualquer desenvolvedor individual carrega na cabeça. Ele se torna o **ativo técnico mais valioso do projeto** — mais que o próprio código, porque código sem o contexto de por que foi escrito daquela forma se deteriora rápido.

**O ponto de virada:** a IA não vai ficar mais inteligente entre uma sessão e outra. O que muda de patamar é o contexto que ela recebe. E é o contexto — não a inteligência do modelo — que determina a qualidade da manutenção.

▸ **COLA PARA FIXAR**

- O ciclo de "corrige aqui, quebra ali" tem causa: o modelo vê **só o fragmento colado**, nunca o sistema.
- A IA erra por **cegueira, não por burrice** — decisão razoável sem contexto gera efeito colateral.
- **Gerar** código novo exige pouco contexto; **manter** exige muito — e é onde menos se fornece.
- Produtos que "funcionam" usam o mesmo modelo que você — com um **system prompt** rico na frente.
- System prompt genérico não conhece as **suas** decisões, convenções, histórico e restrições.
- **Harness** é o system prompt do seu projeto: versionado, específico e que cresce a cada sessão.